

Performance Testing

Date	15 July 2026
Project Name	Smart Lender – AI Powered Loan Approval Prediction System
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing
Target Module / API	Loan Prediction Module (Flask Application)
Test Environment	Local System (Windows 11, Python 3.10, Flask)
Test Date	

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit a valid loan application	10	60	Prediction displayed successfully
2	Multiple users accessing the prediction page simultaneously	25	60	Application remains responsive
3	Continuous prediction requests	50	120	Stable performance with minimal response delay
4	Invalid input testing	10	60	Proper validation messages displayed

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.3 seconds	Pass	Meets expected response time
2	Response Time (Max)	< 5 seconds	3.4 seconds	Pass	Acceptable under load
3	Throughput (Req/sec)	> 20 Req/sec	24 Req/sec	Pass	Good request handling capability
4	Error Rate	< 1%	0.4%	Pass	Very low error rate
5	CPU Utilization	< 80%	62%	Pass	CPU usage remained stable
6	Memory Utilization	< 80%	58%	Pass	Memory usage within acceptable limits

Step 4: Observations & Analysis

Key Findings:

The performance testing showed that the Smart Lender application handled multiple loan prediction requests efficiently. The average response time remained below the expected limit, CPU and memory usage stayed within acceptable ranges, and the application provided accurate loan predictions without significant delays.

Bottlenecks Identified:

Minor response delays were observed when multiple users accessed the application simultaneously. Since the application is currently deployed on a local Flask server, its scalability is limited for handling very high traffic.

Optimization Steps Taken:

The trained XGBoost model was optimized and saved for faster loading during prediction. Data preprocessing was streamlined to reduce processing time, and unnecessary computations were removed to improve application performance. Input validation was also implemented to reduce processing errors and improve reliability.